

Ответы на экзамен: распределённые и облачные БД

2025/2026 · разжёвано с примерами и схемами · подготовка к устному/письменному ответу

Как пользоваться: у каждого вопроса блок «Коротко» (если мало времени), «Разжёвано» (полное понимание), «Пример», схема (где нужна) и фраза «На экзамене скажи».

Содержание (номера вопросов)

1 СУБД · 2 Реляционные/NoSQL · 3 Запросы · 4 Распред. запросы · 5 Фрагментация · 6 Репликация · 7 Индексы · 8 Буфер · 9 Хеш · 10 Кластеризация · 11 Сессии · 12 Транзакции · 13 Ключи · 14 Нормализация · 15 Восстановление · 16 ACID · 17 Распред. транзакции · 18 MAC · 19 GRANT/VIEW · 20 Преимущества · 21 Clustered index · 22 Бэкап · 23 Функции/триггеры · 24 Архитектуры · 25 Клиент-сервер · 26 P2P · 27 Мультибазы · 28 Параллельные · 29 NoSQL/NewSQL · 30 Контроль · 31 Фрагменты при проектировании · 32 Облако · 33 DAC/RBAC · 34 Шифрование · 35 Аудит · 36 Критерии защищённости · 37 Идентификация · 38 Верификация · 39 Угрозы · 40 КИИ

Вопрос 1. Определение СУБД, типы, функции, классификации

Коротко: СУБД — программы, которые хранят данные, дают к ним доступ по правилам, держат целостность при параллельной работе и умеют восстанавливаться после сбоя.

Разжёвано

Без СУБД данные — просто файлы. Два клиента правят один файл — кто-то потеряет изменения. СУБД берёт на себя конкурентный доступ, права, типы, связи (ключи), журнал изменений.

Функции: определение структуры (DDL), манипуляция данными (DML), контроль целостности, буфер/кэш, индексы, транзакции, безопасность, восстановление (WAL/журнал).

Классификации: по модели (реляционные, документные, ключ-значение, графовые, колоночные); по размещению (локальные, клиент-сервер, распределённые, облачные); по нагрузке (OLTP — много коротких транзакций; OLAP — аналитика).

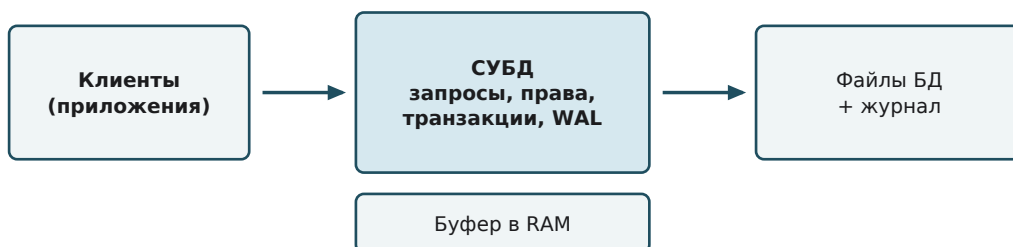


Рис. СУБД между клиентами и файлами данных

Пример. PostgreSQL — реляционная клиент-серверная СУБД. Redis — ключ-значение в памяти. Ваш учебный стенд WEST/EAST — распределённая реляционная схема с репликами.

На экзамене скажи: СУБД = не «таблица Excel», а система с конкурентным доступом, целостностью и восстановлением.

Вопрос 2. Реляционные vs нереляционные БД

Коротко: Реляционные: таблицы, SQL, схема заранее, сильные связи и ACID. NoSQL: гибкая/другая модель, часто горизонтальный масштаб и eventual consistency.

Разжёвано

Реляционная модель: отношения (таблицы), кортежи (строки), атрибуты (столбцы), ключи, нормализация, декларативный SQL. Сильная сторона — строгая структура и JOIN.

Нереляционные (NoSQL): документ (MongoDB), ключ-значение (Redis), широкие колонки (Cassandra), граф (Neo4j). Часто проще масштабировать шардами, схема мягче.

Не «SQL плохо / NoSQL хорошо»: для банковских проводок и жёстких связей — реляционка; для кэша сессий, логов кликов, каталога с разными полями — часто NoSQL.

Критерий	Реляционные	NoSQL
Схема	Жёсткая (таблицы)	Гибкая / своя
Запрос	SQL, JOIN	API / свой язык
Масштаб	Вертикаль + шарды	Часто шарды из коробки
Согласованность	Сильный ACID	Часто eventual

Пример. Интернет-магазин: заказы и оплаты — PostgreSQL; корзина в Redis; поиск товаров — Elasticsearch.

На экзамене скажи: Реляционка = схема + SQL + связи. NoSQL = другая модель под масштаб/гибкость.

Вопрос 3. Общий механизм выполнения запросов

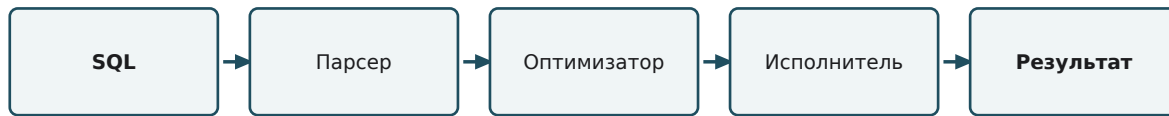
Коротко: SQL → разбор → проверка прав/имён → план (оптимизатор) → исполнение (сканы, JOIN) → ответ.

Разжёвано

Парсер строит дерево запроса. Анализатор сверяет таблицы/столбцы со словарём и правами.

Оптимизатор выбирает план: полный скан или индекс, порядок JOIN, хеш/merge join. Стоимость считают по статистике (число строк, селективность).

Исполнитель читает страницы через буферный пул, применяет фильтры, собирает результат.



общий путь запроса внутри СУБД

Рис. Путь SQL-запроса

Пример. EXPLAIN ANALYZE SELECT * FROM products WHERE sku='SKU-1'; — увидите Index Scan или Seq Scan.

На экзамене скажи: Скажи цепочку: парсер → оптимизатор → исполнитель; план решает, будет ли индекс.

Вопрос 4. Принципы выполнения распределённых запросов

Коротко: Координатор разбивает запрос на куски по узлам/фрагментам, собирает частичные ответы, склеивает.

Разжёвано

Нужен каталог размещения: где лежит фрагмент. Без него некуда слать подзапрос.

Локальная оптимизация на узле + глобальная: минимизировать пересылку данных по сети (лучше фильтровать на месте, чем тащить миллионы строк в центр).

Прозрачность: клиенту желательно писать «как будто одна БД», а маршрутизацию делает слой.



план: куда слать подзапросы → склеить ответы

Рис. Распределённый запрос

Пример. Клиент: заказы WEST и EAST. Координатор шлёт SELECT на оба шарда, UNION результатов. В вашем проекте роль координатора играет FastAPI-клиент.

На экзамене скажи: Распределённый запрос = локальные куски + сеть + склейка; сеть — дорогое место.

Вопрос 5. Фрагментация данных

Коротко: Разбиение отношения на куски (фрагменты) и размещение на узлах. Виды: горизонтальная, вертикальная, смешанная.

Разжёвано

Горизонтальная: строки по предикату (`region='WEST'`). Вертикальная: столбцы (часто + ключ).

Критерии качества: полнота (ничего не потеряли), восстановимость (можно собрать исходное), непересечение (для гориз. — строка в одном фрагменте).

В современных СУБД: партии PostgreSQL, шарды MongoDB/Cassandra, Citus, Vitess.

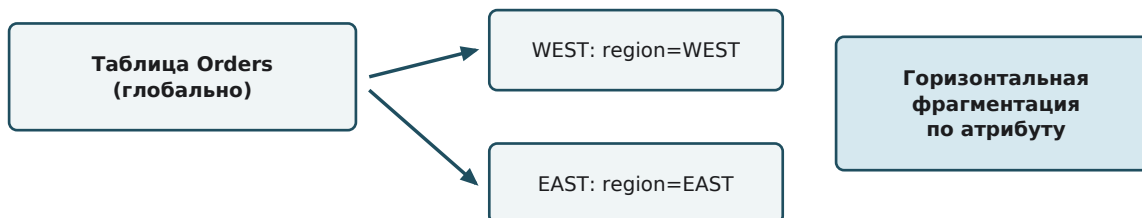


Рис. Горизонтальная фрагментация

Пример. Таблица `shipment_orders`: строки `WEST` только на `west-primary`, `EAST` — на `east-primary`.

На экзамене скажи: Фрагментация = режем данные; репликация = копируем. Не путать.

Вопрос 6. Реплицирование данных

Коротко: Хранение копий на нескольких узлах. Цель: доступность, чтение с копий, географическая близость.

Разжёвано

Синхронная: `primary` ждёт подтверждения `replica` (сильнее consistency, медленнее). Асинхронная: `primary` не ждёт (быстрее, возможен лаг / потеря хвоста при аварии).

Модели: `primary-replica` (мастер-слейв), `multi-master` (сложнее конфликты).

PostgreSQL streaming replication, MySQL binlog, MongoDB replica set.

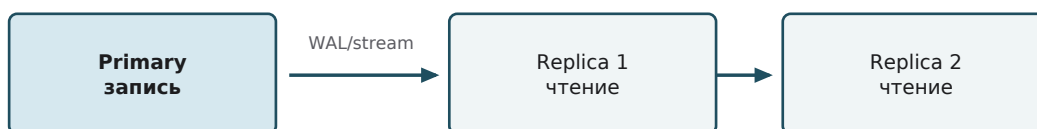


Рис. Репликация `primary` → `replica`

Пример. У вас: `west-primary` пишет, `west-replica` читает через WAL. Убили `primary` — запись падает, чтение с `replica` ещё может жить.

На экзамене скажи: Реплика = копия для чтения/отказоустойчивости, не новый независимый фрагмент.

Вопрос 7. Индексирование

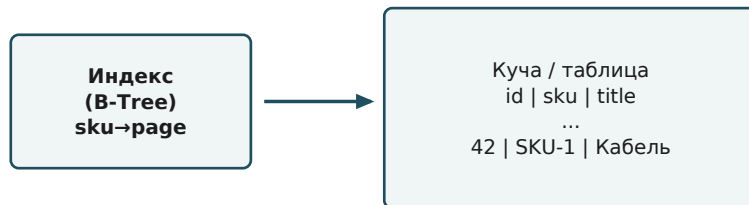
Коротко: Вспомогательная структура «ключ → где лежит строка». Ускоряет поиск, замедляет запись, занимает место.

Разжёвано

B-Tree — универсальный (равенство, диапазоны). Hash — только равенство. GiST/GIN — полнотекст, JSON.

Цена: каждая INSERT/UPDATE обновляет индекс. Лишние индексы вредят записи.

Частичные и составные индексы — под конкретные WHERE.



без индекса — полный скан; с индексом — прыжок к нужной странице

Рис. Индекс указывает на строки

Пример. CREATE INDEX ON products(sku); затем WHERE sku=... идёт по индексу, не Seq Scan.

На экзамене скажи: Индекс — ускоритель чтения ценной записи и места.

Вопрос 8. Буферизация

Коротко: Страницы данных держат в RAM (buffer pool). Чтение/правка сначала в памяти; на диск — позже + через WAL.

Разжёвано

Грязная страница — изменена в буфере, ещё не записана на диск как страница (но запись в WAL уже могла уйти).

Вытеснение: мало места → убирают давно не использованные (LRU и варианты).

Без буфера каждый байт бил бы в диск — СУБД бы «ползла».

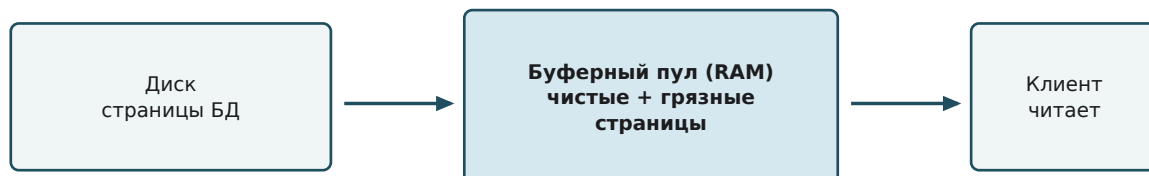


Рис. Буферный пул

Пример. 20 пользователей правят одни и те же заказы — правки крутятся в буфере, диск не дёргают на каждый UPDATE.

На экзамене скажи: Буфер = кэш страниц; долговечность даёт не буфер, а WAL + checkpoint.

Вопрос 9. Хеширование

Коротко: Функция `hash(ключ)` → номер корзины/ячейки. Быстрый точечный доступ по равенству.

Разжёвано

В индексах: `HASH INDEX (PostgreSQL)` — только `=`. В `JOIN`: `hash join` строит таблицу по хешу.

В распределении: `hash(user_id) % N` → номер шарда (равномерное размещение).

Коллизии: разные ключи → одна корзина; решают цепочками/перехешированием.

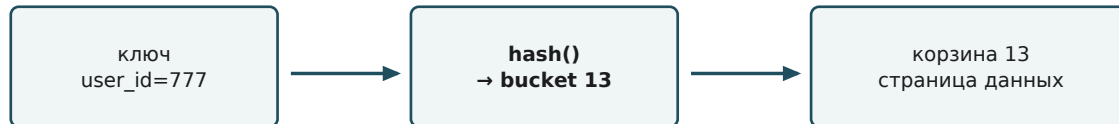


Рис. Хеширование ключа

Пример. Шард = `hash(order_id) % 4` → один из четырёх узлов. Диапазонный поиск по дате хешем не решить — нужен другой ключ/индекс.

На экзамене скажи: Хеш хорош для «равно / шард», плох для «между А и В».

Вопрос 10. Кластеризация данных

Коротко: Физическое упорядочивание строк на диске по ключу (часто по индексу), чтобы соседние по ключу лежали рядом.

Разжёвано

Ускоряет диапазонные запросы и `JOIN` по этому ключу: меньше случайных чтений.

В PostgreSQL `CLUSTER` перестраивает таблицу по индексу (разово; дальше «плывёт» без поддержки).

Не путать с «кластером серверов» (группа машин) — другое значение слова.

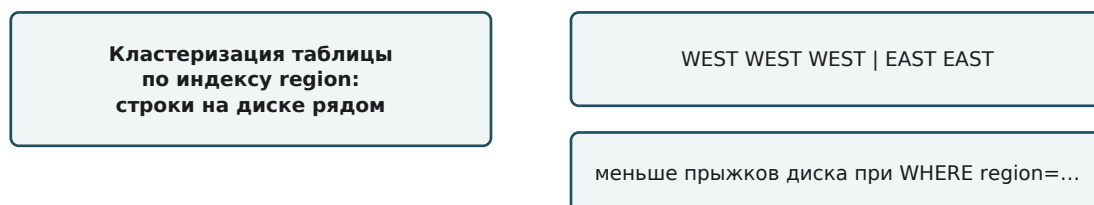


Рис. Кластеризация по ключу

Пример. `CLUSTER orders USING idx_orders_region;` затем выборки по `region` читают подряд идущие страницы.

На экзамене скажи: Кластеризация таблицы = порядок на диске, не путать с failover-кластером.

Вопрос 11. Параллельная работа, сессии и подключения

Коротко: Много клиентов одновременно. Сессия = контекст одного подключения: пользователь, права, транзакция, настройки.

Разжёвано

Подключение (connection) — канал к серверу. Пул соединений экономит дорогие handshake.

Конфликты: два UPDATE одной строки → блокировки/версии; два INSERT — разные id из последовательности.

Изоляция (см. в.16) задаёт, что «видно» чужим сессиям.

Пример. Два кассира бронируют последний товар: одна транзакция RESERVED успешна, вторая получает 409/ошибку остатка.

На экзамене скажи: Сессия = «рабочее место» клиента внутри СУБД на время соединения.

Вопрос 12. Механизм транзакций

Коротко: Транзакция — логическая единица работы: BEGIN ... COMMIT/ROLLBACK. Либо все изменения, либо ни одного.

Разжёвано

Назначение: атомарность сложных операций (деньги списать + зачислить).

Функции: фиксация, откат, изоляция, взаимодействие с журналом (WAL) для durability.

Особенности: длительные транзакции держат блокировки; вложенность/savepoint — частичный откат.

Пример. BEGIN; UPDATE stock...; UPDATE orders...; COMMIT; — если второй UPDATE упал, ROLLBACK вернёт и остаток.

На экзамене скажи: Транзакция = пачка изменений с гарантией «всё или ничего».

Вопрос 13. Ключи: потенциальные, первичные, альтернативные, внешние

Коротко: Ключ однозначно идентифицирует строку. PK — выбранный; альтернативные — остальные кандидаты; FK — ссылка на другой набор.

Разжёвано

Потенциальный (кандидатный) ключ: минимальный набор атрибутов с уникальностью.

Первичный — один выбранный кандидатный. Альтернативный — оставшиеся (например UNIQUE sku при PK id).

Внешний ключ: значение должно существовать в родительской таблице (ссылочная целостность). ON DELETE CASCADE/RESTRICT — политика при удалении родителя.

Тип	Смысл	Пример
Кандидатный	Любой уникальный минимальный	id или sku
Первичный	Выбранный кандидатный	id
Альтернативный	Другой UNIQUE	sku
Внешний	Ссылка на PK/UNIQUE	warehouse_id

Пример. products: PK=id, альтернативный=sku. stock_balances.product_id — FK на products.id.

На экзамене скажи: PK отличает строку; FK связывает таблицы и не даёт «висячих» ссылок.

Вопрос 14. Нормализация и нормальные формы

Коротко: Процедура устранения вредной избыточности: 1НФ → 2НФ → 3НФ (часто цель) → иногда BCNF...

Разжёвано

1НФ: атомарные значения, нет повторяющихся групп.

2НФ: нет частичной зависимости неключевых от части составного ключа.

3НФ: нет транзитивных зависимостей (неключ не зависит от другого неключа).

Нормализация уменьшает аномалии вставки/обновления/удаления. Иногда сознательно денормализуют для скорости/шардинга.

Пример. В проекте region_code в orders дублирует регион склада — отступление от 3НФ ради фрагментации; остальное — 3НФ.

На экзамене скажи: Нормализация = режим зависимости; 3НФ — обычная практическая цель.

Вопрос 15. Определение транзакции. Восстановление

Коротко: Транзакция — атомарная смена состояния БД. Восстановление — вернуть согласованность после сбоя по журналу.

Разжёвано

После commit изменения должны пережить аварию (Durability). До commit — можно откатить.

Восстановление: по WAL redo (накатить зафиксированное) и undo (откатить незафиксированное).

Checkpoint уменьшает объём журнала, который нужно переигрывать при старте.

Пример. Убили процесс PostgreSQL mid-flight: при старте СУБД доигрывает WAL — зафиксированные заказы на месте, незафиксированные исчезли.

На экзамене скажи: Восстановление опирается на журнал, а не на «надежду, что буфер успел сброситься».

Вопрос 16. ACID, точки сохранения, уровни изоляции

Коротко: ACID — Atomicity, Consistency, Isolation, Durability. Savepoint — откат части транзакции. Isolation — насколько видны чужие правки.

Разжёвано

Уровни (от слабого к сильному): Read Uncommitted → Read Committed → Repeatable Read → Serializable.

Read Committed (дефолт PG): не видим чужие незафиксированные; но два SELECT могут дать разное.

Аномалии: грязное чтение, неповторяемое, фантомы — уровни от них по-разному защищают.

SAVEPOINT sp1; ... ROLLBACK TO sp1; — откатить кусок, не всю транзакцию.

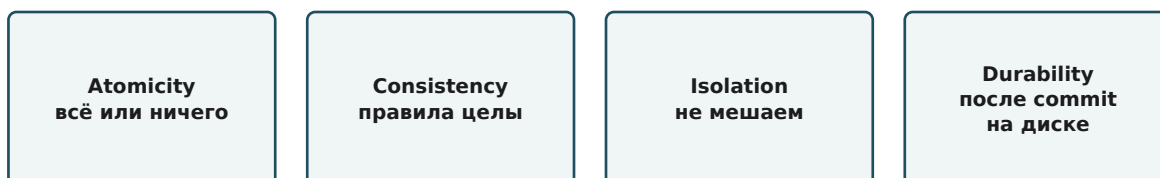


Рис. Свойства ACID

Уровень	Грязное чтение	Неповторяемое	Фантомы
Read Uncommitted	возможно	да	да
Read Committed	нет	да	да
Repeatable Read	нет	нет	часто да*
Serializable	нет	нет	нет

Пример. Касса: READ COMMITTED. Банк с жёсткими инвариантами счетов — часто Serializable или явные блокировки.

На экзамене скажи: ACID на пальцах: всё/ничего; правила целы; не мешаем друг другу; после commit не пропадёт.

Вопрос 17. Транзакции в распределённых БД

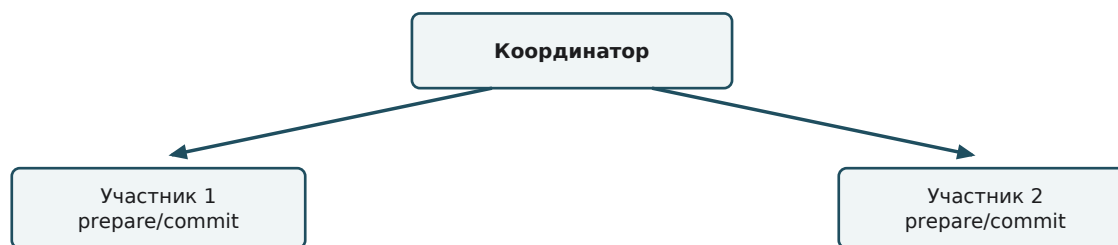
Коротко: Нужно согласовать commit на нескольких узлах. Классика — 2PC (двухфазная фиксация). Цена — задержки и блокировки.

Разжёвано

Фаза prepare: все участники пишут «готов» в журнал. Фаза commit/abort: координатор решает.

Проблемы 2PC: координатор упал в сомнении; блокировки надолго. Есть 3PC, Paxos/Raft-based commit, саги (компенсирующие транзакции).

Часто в практике: локальные ACID + eventual между шардами, или саги вместо глобального 2PC.



2PC: 1) все готовы? 2) commit всем / abort всем

Рис. Двухфазный commit (2PC)

Пример. Перевод денег Москва⇌Владивосток: либо 2PC на оба узла, либо сага «списать → зачислить → компенсировать при сбое».

На экзамене скажи: Распределённая транзакция дороже локальной; 2PC — учебный эталон согласования.

Вопрос 18. Мандатная схема (MAC). Многоуровневая защита

Коротко: Доступ по меткам секретности субъекта и объекта. Пользователь сам себе права «открыть секретно» выдать не может.

Разжёвано

Типичные правила (Белла-Лападула): no read up (не читать выше), no write down (не писать ниже) — против утечки вниз.

Биба — про целостность (зеркально к конфиденциальности).

Многоуровневая защита: сочетание MAC + DAC/RBAC + шифрование + аудит. В Astra Linux мандатные метки — системный уровень.

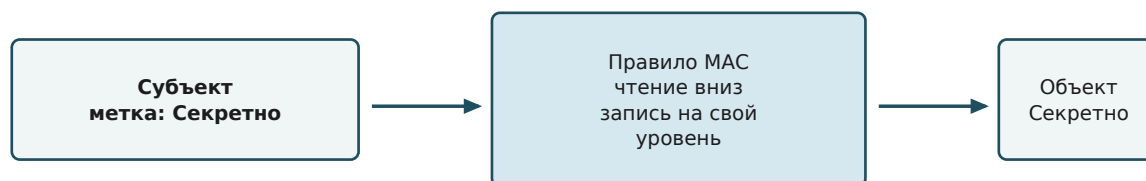


Рис. Мандатный доступ

Пример. Документ «Совершенно секретно» не прочитает пользователь с меткой «секретно», даже если DAC-права дали SELECT.

На экзамене скажи: MAC = метки сверху; DAC = владелец раздаёт; вместе = оборона в глубину.

Вопрос 19. Представления и защита. GRANT / REVOKE

Коротко: VIEW — виртуальная таблица-запрос. Через неё дают «окно» на часть данных. GRANT выдаёт права, REVOKE забирает.

Разжёвано

Представление скрывает столбцы/строки (зарплаты других, чужие регионы).

GRANT SELECT ON v_west_orders TO app_west; — приложению не нужна вся таблица.

REVOKE — отзыв. WITH GRANT OPTION — право передоверять (осторожно).

Дополнительно: RLS (row level security) в PostgreSQL — политика «видишь только свои строки».

Пример. VIEW без столбца salary + GRANT SELECT на VIEW аналитику. Таблицу employees напрямую не даём.

На экзамене скажи: VIEW + GRANT = минимально необходимые права, не «всем ALL на всё».

Вопрос 20. Преимущества распределённых систем. Примеры

Коротко: Масштаб, близость к пользователю, отказоустойчивость, автономия площадок. Цена — сложность согласованности и сети.

Разжёвано

Преимущества: горизонтальный рост, локальность данных (latency), независимость контуров, переживание отказа узла.

Примеры: CDN+геореплики БД, банковские региональные контуры, ваш WEST/EAST складской стенд, Cassandra-кластеры, Google Spanner.

Пример. Склад во Владивостоке пишет в EAST локально — не гоняет каждую транзакцию в Москву.

На экзамене скажи: Распределённость даёт масштаб и живучесть, но усложняет согласованность.

Вопрос 21. Кластеризованный и некластеризованный индекс

Коротко: Кластеризованный: порядок строк таблицы $\approx$ порядок индекса (часто PK в SQL Server). Некластеризованный: отдельная структура указателей, куча/таблица может быть в другом порядке.

Разжёвано

В SQL Server/MySQL InnoDB: clustered по PK — данные «в листьях» индексного дерева.

В PostgreSQL нет классического clustered index как в SQL Server; есть вторичные индексы + команда CLUSTER.

Некластеризованных индексов много; кластеризованный (где есть) обычно один.

Пример. SQL Server: PK на OrdersId кластеризует таблицу. Индекс на CustomerId — nonclustered, хранит ключ $\rightarrow$ указатель на строку.

На экзамене скажи: Clustered $\approx$ «таблица лежит в порядке индекса»; nonclustered $\approx$ «отдельный указатель».

Вопрос 22. Резервирование (бэкап) БД

Коротко: Копии для восстановления после потери/порчи. Полный, дифференциальный, инкрементальный; плюс архив WAL для PITR.

Разжёвано

Логический дамп (pg_dump) — SQL/архив объектов. Физический — копирование файлов данных (+ WAL).

PITR: база + непрерывный архив журналов $\rightarrow$ откат на момент времени.

Правило 3-2-1: 3 копии, 2 носителя, 1 offsite. Регулярно проверять restore!

Пример. ./scripts/make_dump.sh $\rightarrow$ dump/west_warehouse.sql. Это логический бэкап учебных шардов.

На экзамене скажи: Бэкап без проверки восстановления — ложное спокойствие.

Вопрос 23. Хранимые функции и триггеры

Коротко: Функции/процедуры — код в СУБД. Триггер — автоматическая реакция на INSERT/UPDATE/DELETE.

Разжёвано

Плюсы: логика рядом с данными, меньше кругов по сети. Минусы: сложнее версионировать, отладка, сюрпризы производительности.

Триггеры опасны рекурсией и скрытыми побочными эффектами; в проде часто ограничивают.

Пример. Триггер AFTER UPDATE ON orders пишит строку в audit_log. Функция calc_vat(amount) считается на сервере.

На экзамене скажи: Триггер = «если событие — сделай»; функция = «вызови по имени».

Вопрос 24. Классификация архитектур распределённых БД

Коротко: Клиент-сервер, одноранговые (P2P), мультибазовые, параллельные, облачные/DBaaS; также по однородности и автономности.

Разжёвано

Однородные (одна СУБД везде) vs неоднородные (разные движки под общим слоем).

С общей схемой vs федеративные (каждый узел со своей схемой + посредник).

Пример. Ваш стенд: однородная PostgreSQL, клиент-сервер к каждому узлу, горизонтальные фрагменты + реплики.

На экзамене скажи: Назови 3–4 типа и чем отличаются по «кто главный» и «общая ли схема».

Вопрос 25. Клиент-серверные БД

Коротко: Клиенты шлют запросы на сервер СУБД; сервер владеет данными, журналом, правами.

Разжёвано

Толстый/тонкий клиент: логика на клиенте или на сервере (процедуры).

Масштаб: один сервер → вертикаль; дальше реплики/шарды.

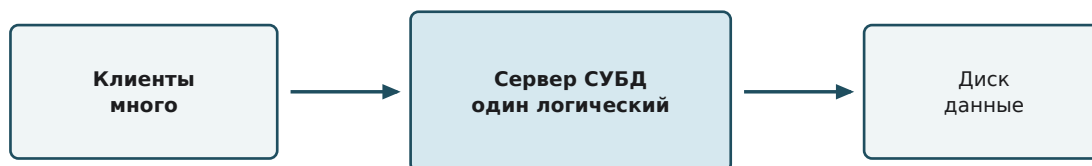


Рис. Клиент-сервер

Пример. psql / приложение → PostgreSQL на порту 5432. Классика.

На экзамене скажи: Клиент-сервер = центральный сервер данных + много клиентов.

Вопрос 26. Одноранговые (P2P) БД

Коротко: Узлы равноправны: каждый может хранить/отдавать данные, нет единственного обязательного центра.

Разжёвано

Плюсы: живучесть, нет единой точки отказа. Минусы: сложный поиск данных, согласованность, безопасность.

Примеры идей: распределённые хеш-таблицы, некоторые blockchain/ledger-подходы, P2P file systems.



Рис. Одноранговая сеть узлов

Пример. Сеть узлов, где запрос «кто хранит ключ К?» идёт по протоколу маршрутизации, а не на фиксированный master.

На экзамене скажи: P2P = равные узлы; клиент-сервер = выделенный сервер.

Вопрос 27. Мультибазовые СУБД

Коротко: Слой над несколькими уже существующими БД (часто разными). Глобальная схема/посредник, локальные СУБД автономны.

Разжёвано

Задача: единый запрос к наследству Oracle + PostgreSQL + файлы без полной миграции.

Сложности: глобальная оптимизация, типы, транзакции между разными движками.

Пример. Предприятие: FDW в PostgreSQL смотрит на внешнюю MySQL как на foreign table — упрощённый мультибазовый приём.

На экзамене скажи: Мультибаза = «зонтик» над чужими автономными БД.

Вопрос 28. Параллельные СУБД

Коротко: Много процессоров/узлов ускоряют один запрос или поток транзакций: shared-memory, shared-disk, shared-nothing.

Разжёвано

Shared-nothing хорошо масштабируется шардами (каждый узел со своим диском).

Параллелизм: межзапросный (много мелких) и внутризапросный (один SELECT на куски).

Пример. Greenplum/ClickHouse/параллельный Seq Scan в PG на нескольких CPU — варианты параллелизма.

На экзамене скажи: Параллельная СУБД жмёт на скорость за счёт многих вычислителей.

Вопрос 29. NoSQL и NewSQL

Коротко: NoSQL — нереляционные модели под масштаб/гибкость. NewSQL — горизонтальный масштаб при сохранении SQL+ACID.

Разжёвано

NoSQL: отказ от жёсткого JOIN/схемы ради шардов и availability (часто CAP-компромисс).

NewSQL (Spanner, CockroachDB, TiDB): шарды + распределённые транзакции + привычный SQL.

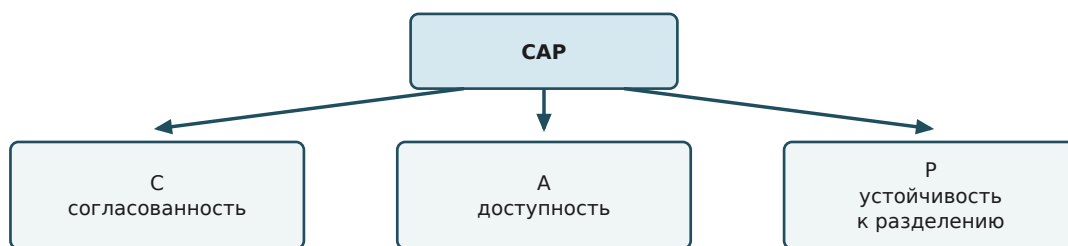


Рис. CAP-теорема

Пример. Cassandra пишет логи по миру (NoSQL). CockroachDB — SQL с репликами диапазонов (NewSQL).

На экзамене скажи: NoSQL ≠ «без SQL навсегда»; NewSQL = SQL+ACID+шарды.

Вопрос 30. Контроль в распределённых БД

Коротко: Контроль размещения, согласованности реплик, транзакций, доступа, аудита и производительности по узлам.

Разжёвано

Каталог/метаданные: где фрагмент. Мониторинг лага репликации. Кворумы. Политики безопасности единообразно.

Без контроля «левая рука не знает, что пишет правая» — расходятся копии и права.

Пример. Алёрт: replication lag > 30s на west-replica; запрет записи при рассинхроне критичного контура.

На экзамене скажи: Контроль = видимость состояния всех узлов + правила реакции.

Вопрос 31. Фрагментирование при проектировании: виды и принципы

Коротко: На этапе проектирования решают: по какому атрибуту резать, куда класть, как обеспечить полноту/восстановимость.

Разжёвано

Принципы: локальность ссылок (данные рядом с приложениями), равномерность нагрузки, минимизация распределённых JOIN, безопасность периметра (как у вас — `region_code`).

Виды те же: H/V/смешанная. Производная фрагментация: `items/events` едут за `orders`.

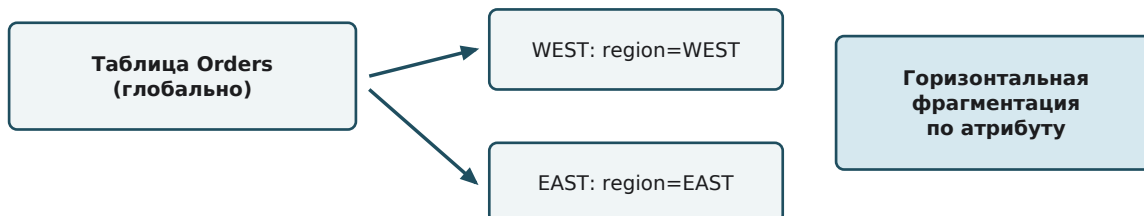


Рис. Горизонтальная фрагментация

Пример. Критерий: `region_code`. `Orders WEST` не должны попадать на `EAST` — и по нагрузке, и по контуру КИИ.

На экзамене скажи: Сначала критерий разреза, потом узлы, потом проверка полноты.

Вопрос 32. Сетевые и облачные БД

Коротко: Облако: ресурсы по сети как сервис (`IaaS/PaaS/DBaaS`). Причины: экономика, эластичность, скорость запуска.

Разжёвано

`DBaaS`: Amazon RDS, Cloud SQL, Yandex Managed PostgreSQL — патчи, бэкапы, реплики «из коробки».

Риски: зависимость от провайдера, каналы, модель ответственности (`shared responsibility`), требования КИИ/локализации.

Частный облачный контур на своей инфраструктуре — для чувствительных объектов.

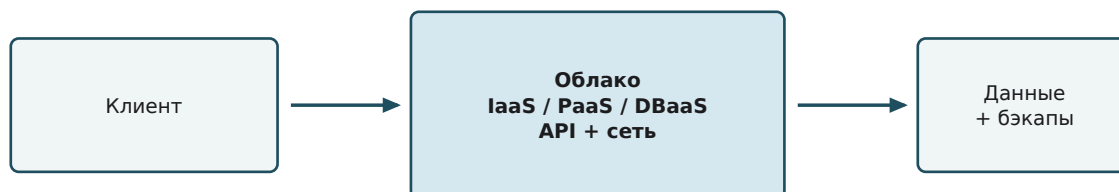


Рис. Облачный доступ к БД

Пример. Стартап поднимает Managed PostgreSQL за час вместо покупки железа. Банк КИИ — чаще свой контур.

На экзамене скажи: Облако = удобство и масштаб; для КИИ смотри регуляторику и периметр.

Вопрос 33. Дискреционное (DAC) и ролевое (RBAC) vs мандатное (MAC)

Коротко: DAC: владелец объекта раздаёт права. RBAC: права через роли. MAC: метки обязательной политики, пользователь не обойдёт.

Разжёвано

DAC гибок, но риск ошибок («выдали всем»). RBAC удобен в организациях (роль cashier, auditor).

MAC жёстче: метка сессии и объекта. Сравнение: DAC/RBAC — «кто кому разрешил»; MAC — «какая категория секретности».

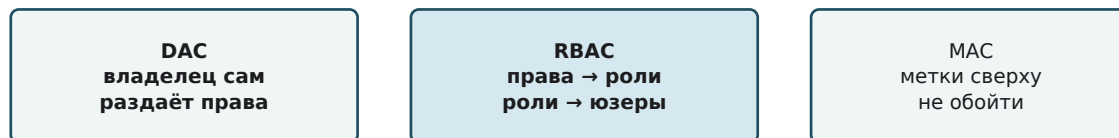


Рис. DAC / RBAC / MAC

Пример. GRANT SELECT TO role_report (RBAC). Владелец таблицы дал другу ALL (DAC). Мандатная метка «CC» не читается с «C» (MAC).

На экзамене скажи: Назови три модели и кто «хозяин» решения о доступе.

Вопрос 34. Шифрование данных и каналов. Управление ключами

Коротко: At rest — на диске; in transit — TLS в сети. Ключи хранят в KMS/HSM, ротация, разделение ролей.

Разжёвано

Transparent Data Encryption (TDE), шифрование табличных пространств, колоночное (pgcrypto) для особо чувствительных полей.

Канал: TLS между клиентом и СУБД, TLS/VPN на репликации.

Ключи не кладут в git. Компрометация ключа = компрометация шифротекста.

Пример. PostgreSQL sslmode=require; бэкапы в S3 с server-side encryption; пароли только хеш (bcrypt/bcrypt), не шифр обратимый.

На экзамене скажи: Шифр без управления ключами бесполезен; канал и диск — разные задачи.

Вопрос 35. Аудит безопасности БД

Коротко: Журналирование кто/что/когда/откуда сделал. В распределённой и облачной среде — свод логов со всех узлов + облачный trail.

Разжёвано

Аудит DDL/DCL, входов, доступа к чувствительным таблицам, ошибок аутентификации.

Облако: CloudTrail/аналоги + логи СУБД. Важно единое время (NTP) и защита самих логов от подчистки.

Пример. shipment_events у вас — прикладной журнал статусов; плюс нужен системный audit log входов и GRANT.

На экзамене скажи: Аудит = неотрицаемое «кто трогал»; логи тоже защищать.

Вопрос 36. Критерии защищённости облака и отдельных БД

Коротко: Оценивают: изоляцию арендаторов, шифрование, управление ключами, аудит, обновления, резервы, соответствие стандартам/регулятору.

Разжёвано

Для БД: разграничение доступа, минимальные привилегии, патчи, hardening (лишние порты/роли), WAF/сеть, бэкапы.

Для облака: сертификации провайдера, регион хранения, ответственность сторон, мониторинг misconfig.

Пример. Чек-лист: TLS обязателен, superuser только админам, бэкап проверен restore-тестом, лаг реплики под контролем.

На экзамене скажи: Защищённость = набор измеримых контролей, не одна «галочка шифрование».

Вопрос 37. Идентификация и аутентификация. Распред./облако

Коротко: Идентификация — кто ты (логин). Аутентификация — докажи (пароль, сертификат, MFA, SSO).

Разжёвано

В распределённой среде единый IdP (LDAP/AD/Keycloak) лучше кучи локальных паролей.

Облако: IAM-роли, временные токены, federation. Сервисные учётки отдельно от людей.

После входа — авторизация (уже DAC/RBAC/MAC).

Пример. Приложение ходит в БД под ролью `app_west` с паролем из vault; админ — по SSO + MFA.

На экзамене скажи: Сначала «кто» и «докажи», потом «что можно».

Вопрос 38. Верификация данных в распределённых и облачных ИС

Коротко: Проверка, что данные целы, не искажены, соответствуют правилам и ожидаемому источнику.

Разжёвано

Целостность: PK/FK, CHECK, ограничения предметной области, хэш/подпись пакетов.

Между узлами: сверка контрольных сумм реплик, антиэнтропия, quorum reads.

Облако: валидация на входе API, схемы сообщений, недоверие к клиентскому вводу.

Пример. После репликации сравнить `count(*)` и `checksum` агрегата на `primary` и `replica`; `CHECK status IN (...)`.

На экзамене скажи: Верификация = доказать корректность, а не «надеяться, что доехало».

Вопрос 39. Угрозы распределённых БД. Защита каналов репликации и удалённого доступа

Коротко: Угрозы: перехват канала, подмена узла, несанкционированный доступ, DoS, рассинхрон, инсайдер, инъекции.

Разжёвано

Репликация: только TLS/VPN, аутентификация узлов, ограничение source IP, отдельная сеть.

Удалённый доступ: jump-host, MFA, запрет прямого 5432 в интернет, allowlist.

Прикладные: SQL-инъекции — параметризация; минимальные права приложения.

Пример. Стриминг репликации PostgreSQL только по private network + сертификаты; SSH-туннель для админа.

На экзамене скажи: Самое слабое место часто канал и учётки, не «алгоритм AES».

Вопрос 40. Контроль ИБ в распределённой среде. Защита БД объектов КИИ

Коротко: Для КИИ — повышенные требования: сегментация, учёт, аудит, устойчивость, соответствие приказам/ФЗ, иногда запрет иностранных облаков.

Разжёвано

Меры: периметр и зоны, мандатный доступ где требуется, резервирование, регулярные учения restore, контроль поставщиков, журналы, реагирование на инциденты.

Распределённость КИИ: каждый контур (как WEST/EAST) автономен, но политики безопасности едины; критерий фрагментации может совпадать с периметром безопасности.

Пример. Учебный критерий `region_code` = периметр площадки: чужой регион → 403, даже если сеть «технически пускает».

На экзамене скажи: КИИ: безопасность и непрерывность важнее удобства публичного облака.

Шпаргалка на 30 секунд перед экзаменом

- СУБД ≠ файл: конкурентный доступ + права + восстановление.
- Запрос: парсер → оптимизатор → исполнитель.
- Фрагмент = кусок данных; реплика = копия.
- ACID; 2PC для распределённого commit.
- Индекс ускоряет чтение; буфер — RAM-кэш страниц; WAL — долговечность.
- DAC / RBAC / MAC — три разные логики доступа.
- Облако удобно; КИИ требует периметр, аудит, свой контур.

Файл: Ответы_на_экзамен_РБД.pdf · пересборка: `python _build/build_exam_answers_pdf.py`